RAG & STREAMLIT

AI Engineering Learning Notes

1. RAG (Retrieval-Augmented Generation)

What is RAG?

RAG stands for Retrieval-Augmented Generation. It is an AI technique that combines information retrieval with a Large Language Model (LLM). Instead of depending only on the knowledge stored in the model, RAG retrieves relevant information from an external knowledge source and provides that information to the LLM to generate a more accurate and context-aware answer.

Why is RAG Used?

- To answer questions using private or custom documents.
- To reduce incorrect or hallucinated answers by giving the model relevant context.
- To use updated information without retraining the entire LLM.
- To build document-based chatbots and question-answering systems.
- To connect LLMs with databases, PDFs, websites, and other knowledge sources.

Basic RAG Workflow

1. Load Data: Collect documents such as PDFs, text files, web pages, or database records.

2. Split Documents: Break large documents into smaller chunks so that relevant information can be retrieved efficiently.

3. Create Embeddings: Convert text chunks into numerical vector representations called embeddings.

4. Store Vectors: Store embeddings in a vector database such as FAISS, Chroma, Pinecone, or another vector store.

5. User Query: The user asks a question in natural language.

6. Retrieve Relevant Chunks: The query is converted into an embedding and similar document chunks are retrieved.

7. Augment the Prompt: The retrieved information is added to the prompt as context.

8. Generate Answer: The LLM uses the question and retrieved context to generate the final response.

Important RAG Components

Documents: The original knowledge source, such as PDFs, notes, websites, or database records.

Chunking: Splitting documents into smaller pieces for efficient retrieval.

Embeddings: Numerical representations of text that capture semantic meaning.

Vector Database: Stores embeddings and helps find semantically similar content.

Retriever: Finds the most relevant chunks for a user query.

LLM: Uses the retrieved context to generate the final natural-language answer.

Prompt: Combines the user question with retrieved context and instructions.

Simple RAG Example

Suppose a company has an internal employee handbook in PDF format. A user asks, "How many casual leaves can an employee take?" A RAG system searches the handbook for the relevant section, retrieves the appropriate text, and gives that text to the LLM. The LLM then generates an answer based on the retrieved company policy.

RAG vs Traditional LLM

Feature	Traditional LLM	RAG
External Knowledge	Limited	Can use external knowledge
Private Documents	Not directly available	Can retrieve from private data
Updated Information	Needs updated knowledge/model	Can retrieve current data
Hallucination Risk	Can be higher	Can be reduced with relevant context

2. Streamlit

What is Streamlit?

Streamlit is an open-source Python framework used to quickly build interactive web applications, especially for data science, machine learning, AI, and Python projects. It allows developers to create a user interface directly from Python without needing to write a large amount of HTML, CSS, or JavaScript.

Why is Streamlit Used?

- To create simple and interactive AI/ML application interfaces.
- To demonstrate machine learning models.
- To build dashboards and data applications.
- To create prototypes quickly.
- To build interfaces for RAG and LLM applications.
- To share Python-based projects through a web interface.

Basic Streamlit Example

```
import streamlit as st

st.title("My First Streamlit App")

name = st.text_input("Enter your name")

if st.button("Submit"):
    st.write("Hello", name)
```

Common Streamlit Components

st.title(): Displays the main title.

st.header(): Displays a section heading.

st.write(): Displays text, numbers, objects, and other content.

st.text_input(): Takes text input from the user.

st.number_input(): Takes numeric input.

st.button(): Creates a clickable button.

st.selectbox(): Creates a dropdown selection.

st.checkbox(): Creates a checkbox.

st.file_uploader(): Allows users to upload files.

st.session_state: Stores values between Streamlit reruns, useful for counters and application state.

Streamlit Session State

Streamlit reruns the Python script when the user interacts with widgets. st.session_state is used to preserve values across these reruns. For example, it can be used to maintain a counter, login state, selected values, or chat history.

```
if "count" not in st.session_state:
    st.session_state.count = 0

if st.button("Increase"):
    st.session_state.count += 1

st.write("Count:", st.session_state.count)
```

Streamlit + RAG

Streamlit can be used as the frontend for a RAG application. The RAG pipeline handles document loading, chunking, embeddings, retrieval, and LLM generation, while Streamlit provides the user interface.

1. User uploads a PDF using st.file_uploader().
2. The application extracts and splits the document into chunks.
3. Embeddings are generated for the chunks.
4. The embeddings are stored in a vector database.
5. The user enters a question in a text input.
6. The RAG pipeline retrieves relevant chunks.
7. The LLM generates an answer using the retrieved context.
8. Streamlit displays the answer to the user.

Typical RAG + Streamlit Project Architecture

User → Streamlit UI → Query Processing → Embedding Model → Vector Database → Retriever → Prompt + Retrieved Context → LLM → Answer → Streamlit UI

Key Takeaways

- RAG connects LLMs with external knowledge.
- Embeddings represent text as vectors and help with semantic search.
- Vector databases store and retrieve similar embeddings.
- Streamlit provides a simple Python-based interface for AI applications.
- RAG and Streamlit together can be used to build document chatbots and AI assistants.

End of Notes